Calculadora de Área de un Círculo con Diseño Modular Top-Down en C++

Cheheid, Malek

Laboratorio de Programación (LPR)

5.º Año – 3.º Grupo A

E.E.S.T. N.º 1 «Eduardo Ader» – Vicente López

Prof. Ganduglia, Yamil

2026

Repositorio: https://github.com/malekcheheid-art/CALCULOAREA

Resumen

En este trabajo escribí un programa en C++ que calcula el área de un círculo a partir de su radio. Para resolverlo usé un diseño Top-Down, que consiste en dividir un problema grande en partes más pequeñas y fáciles de resolver. El programa quedó organizado en cuatro bloques: el prototipo de la función, la llamada desde el programa principal, la impresión del resultado y la definición de la función que hace el cálculo. Para obtener el área usé la fórmula $A = \pi \times r^2$, con el valor de π declarado como constante para que nadie pueda cambiarlo por error. También agregué una validación que rechaza los radios negativos o iguales a cero, porque esas medidas no describen ninguna figura real.

Palabras clave: programación en C++, diseño Top-Down, modularización, funciones, constantes, validación de datos

1. Introducción

En esta actividad tuve que escribir un programa que calcule el área de un círculo. A primera vista parece un ejercicio fácil, pero al empezar a programarlo me di cuenta de que la pregunta importante no era la fórmula, sino cómo estructurar el código para que quede ordenado y fácil de entender.

La solución que elegí fue dividir el programa en partes. En lugar de escribir todo dentro de main(), separé el cálculo en una función aparte. A este forma de trabajar se la llama diseño Top-Down, porque se empieza por el problema general y después se baja a los detalles. Así, el programa quedó formado por módulos chicos, cada uno con una sola responsabilidad.

Trabajé en Windows 10/11, usando el compilador GNU g++ y la terminal integrada de Visual Studio Code. Pude compilar en mi propia computadora, así que no tuve que usar OnlineGDB.

El código y las evidencias de este trabajo están en mi repositorio de GitHub: https://github.com/malekcheheid-art/CALCULOAREA.

2. Marco teórico

2.1 Diseño Top-Down

Para este trabajo usé el diseño Top-Down, que es una forma de resolver problemas empezando por lo general y bajando después a los detalles. Cada vez que el problema es muy grande, lo divido en partes más pequeñas hasta que cada parte quede fácil de resolver por separado. En este programa lo usé para que main() se encargue de pedir el radio y mostrar el resultado, mientras el cálculo matemático queda en una función aparte.

2.2 Modularización

Modularizar es dividir el programa en módulos o funciones. Yo lo hice así: en vez de escribir todo seguido dentro de main(), separé cada tarea en una función. Con eso gané tres cosas. Primera, los errores se encuentran más rápido, porque el problema queda aislado dentro de un módulo y no tengo que revisar todo el código. Segunda, el mismo módulo lo puedo volver a usar en otro programa sin escribirlo de nuevo. Tercera, si trabajo con otra

persona, cada una se ocupa de una parte sin meterse en la del otro.

2.3 Prototipo de función

El prototipo es la declaración que escribo antes de main(). Solo dice cómo se llama la función, qué tipo de dato devuelve y qué parámetros recibe. No hace nada por sí misma. Su trabajo es avisarle al compilador que esa función existe, para que no se quede sin entender cuando la encuentra dentro de main().

2.4 Retorno (return)

El retorno es el valor que la función le devuelve al programa principal cuando termina de trabajar. En mi caso, la función recibe el radio, hace el cálculo y devuelve el área con la sentencia return. Así main() recibe el resultado y ya puede mostrarlo.

2.5 Constantes (const)

Una constante es un valor que queda bloqueado en la memoria y que no se puede cambiar durante toda la ejecución del programa. El caso clásico es el número π, que es una constante universal de la geometría. Yo lo declaré como const double PI para que el compilador no le permita escribir encima. Si alguien intentara cambiarlo, el programa directamente no compilaría.

3. Metodología

Trabajé en cinco etapas. Primero leí el enunciado y detecté que el problema se podía separar en cuatro bloques. Segundo, armé la estructura de carpetas que pide la consigna, que se muestra en la Tabla 1. Tercero, escribí el código en src/main.cpp. Cuarto, lo compilé y lo ejecuté con g++ para comprobar que funcionaba. Quinto, armé las capturas y escribí este informe. El código completo, tal como lo entregué, está en el Apéndice A.

3.1 Estructura del repositorio

Tabla 1

Estructura de carpetas de la actividad

Archivo o carpeta	Descripción
.gitignore	Ignora la carpeta .vscode y los archivos de compilación
LICENSE	Licencia MIT del trabajo
README.md	Documentación general del programa
docs/	Documentación: este informe y la consigna
src/	Código fuente del programa en C++
capturas/	Capturas de compilación y del diagrama de flujo

3.2 Compilación y ejecución

Desde la terminal integrada de VS Code, ubicada en la carpeta CALCULOAREA, escribí estos dos comandos:

```
g++ src/main.cpp -o src/area.exe
.\src\area.exe
```

El primero crea el ejecutable (area.exe) a partir del código fuente. El segundo pone el programa a correr, que después me pide el radio por pantalla.

4. Desarrollo del programa

En src/main.cpp resolví los cuatro bloques que pide la consigna. A continuación explico cada uno.

4.1 Bloque 1: prototipo de la función

Escribí el prototipo antes de main() para que el compilador conozca la función antes de encontrarla:

```
double calcularAreaCirculo(double radio);
```

4.2 Bloque 2: llamada a la función

Desde main() llamo al módulo pasándole el radio y guardo lo que devuelve en una variable nueva llamada areaFinal:

```
double areaFinal = calcularAreaCirculo(radioEstudiante);
```

4.3 Bloque 3: salida de datos

Después muestro el resultado en pantalla, junto con el radio que había ingresado:

```
cout << "Radio ingresado: " << radioEstudiante << " cm" << endl;
cout << "Area del circulo: " << areaFinal << " cm^2" << endl;
```

4.4 Bloque 4: definición de la función

Acá escribo el módulo que hace el cálculo. Usa la constante PI y una variable local, y devuelve el resultado:

```
double calcularAreaCirculo(double r) {
    const double PI = 3.1415926535;
    double area = PI * r * r;
    return area;
}
```

El parámetro r es una variable local: existe solo dentro del módulo y nadie más lo puede tocar. Gracias a eso, el radio que se usa en el cálculo no se puede modificar por accidente desde otra parte del programa.

4.5 Filtro de consistencia

Antes de llamar al módulo, el programa revisa que el radio tenga sentido. Si no lo tiene, el programa se frena ahí mismo:

```
if (radioEstudiante <= 0) {
    cout << "[ERROR] El radio debe ser un valor positivo y mayor a cero." << endl;
    return 1;
}
```

Además, el programa devuelve 0 cuando todo sale bien y 1 cuando hay error. Así cualquiera que lo ejecute sabe si funcionó o no.

5. Resultados

El programa compiló sin errores ni advertencias. Lo probé con tres radios distintos: 3 cm, 8 cm y 20 cm, que cubren los tres tamaños de diana del bonus. Los áreas obtenidas se muestran en la Tabla 2. En la Figura 1 se ve una de esas corridas, con el radio de 8 cm. El flujo completo del programa está representado en el diagrama de la Figura 2.

Tabla 2

Resultados de las tres pruebas

Radio (cm)	Área (cm²)	Diana que se dibuja
3	28.27	DIANA MINI
8	201.06	DIANA ESTÁNDAR
20	1256.64	DIANA GIGANTE

Figura 1

Compilación y ejecución del programa en la terminal de Windows.

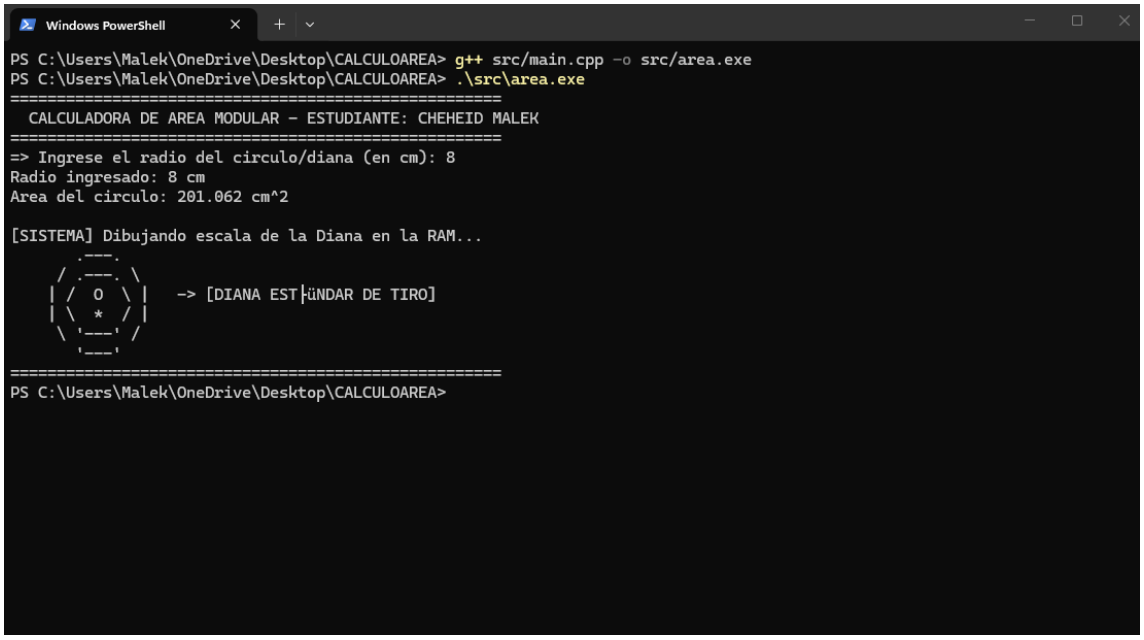
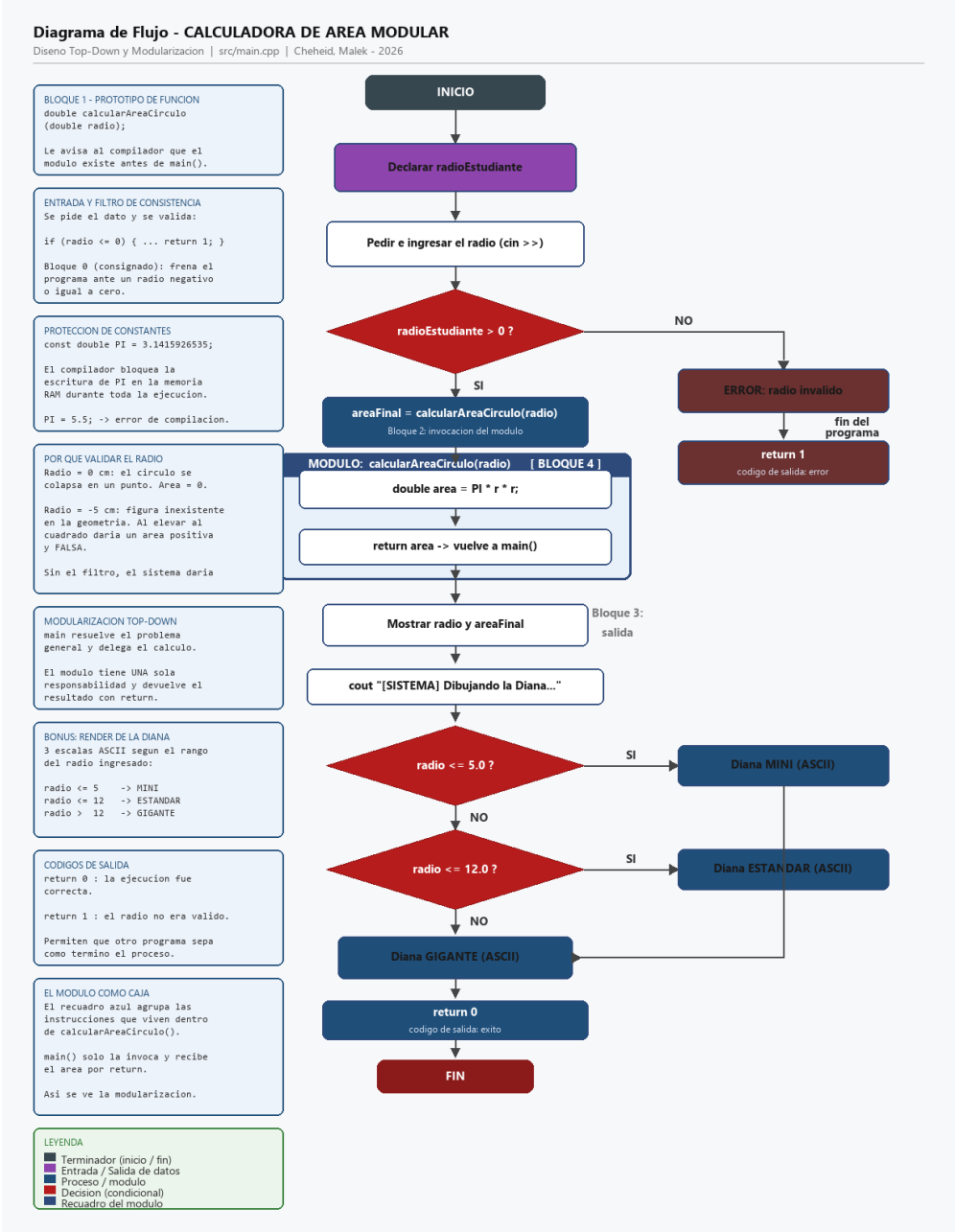


Figura 2

Diagrama de flujo del programa.



6. Cuestionario técnico de control

Pregunta 1. El prototipo de la función

Si borráramos la línea 15 (double calcularAreaCirculo(double radio);), ¿por qué creés que el compilador no sabría qué hacer cuando intente leer la función main?

Lo que pasaría es que el compilador lee el archivo de arriba hacia abajo, en una sola pasada, y no puede mirar hacia adelante. Cuando llega a la línea donde llamo a calcularAreaCirculo, todavía no leyó ninguna definición de esa función. Lo único que tiene a mano es el prototipo que escribí unas líneas arriba. Sin él no podría saber ni el nombre de la función, ni qué tipo de dato devuelve, ni cuántos parámetros espera, y no tendría otra salida que mostrar un error de compilación. Por eso el prototipo no es un detalle de estilo: es la forma de avisarle al compilador que la función existe antes de usarla, y así él puede revisar cada llamada en el momento justo.

Pregunta 2. El filtro de consistencia

¿Qué problema físico o geométrico ocurriría si intentáramos calcular el área de un círculo con un radio de 0 cm o de -5 cm? ¿Por qué es importante controlar esto?

Según yo, con un radio de 0 cm no hay círculo: la circunferencia se reduce a un solo punto y no queda superficie dentro. El área daría 0, que es un número correcto pero engañoso, porque el usuario creyó haber obtenido una figura útil. El caso del radio negativo es peor. Una distancia negativa no existe en la geometría, porque las distancias nunca son negativas. Igual, si no lo controlamos, la cuenta se hace igual: $(-5)^2$ da 25, y el programa devolvería un área positiva de 78.54 cm^2 para algo que jamás existió. El fallo no aparece como un número negativo que me da cuenta de inmediato, sino como un resultado normal y por eso equivocado. Eso es lo más peligroso: un error que no avisa que está ocurriendo. Para mí lo importante es validar porque revisar lo que ingresa el usuario es una de las buenas prácticas básicas al programar: cuida que los datos sean reales, evita que un dato dañado se propague al resto del sistema y hace que el programa se comporte igual siempre.

Pregunta 3. Proteger constantes en RAM

¿Por qué considerás que es más seguro para un sistema crítico (como el software de un avión o un sistema médico) declarar valores como inmutables (const) en lugar de usar variables comunes?

Para mí, en un sistema crítico un valor mal puesto no produce un simple error de cuenta: puede tener consecuencias graves. Usar const no es una cuestión de estítica, es una protección que se aplica al compilar. Cuando escribo const double PI, el compilador guarda ese valor en un lugar de la memoria que solo admite lectura y, si alguien intenta escribir PI = 5.5;, deja compilar el programa. Eso es muy distinto a una variable común, que se puede reescribir en cualquier momento, por un error o por un camino del programa que nadie previó, sin que nada avise. En avión, medicina o finanzas, esa diferencia separa un error que se detecta al compilar de un defecto que llega hasta el vuelo o hasta el paciente. En resumen, const convierte una regla del trabajo en algo que el lenguaje obliga a cumplir.

7. Conclusiones

Con este trabajo entendí que modularizar no es alargar el código, sino ordenar el problema antes de escribirlo. Un programa que está todo junto dentro de main(), con varias responsabilidades mezcladas, quedó dividido en cuatro piezas chicas y chacune con una tarea clara.

Aprendí tres cosas concretas. La primera, para qué sirve el prototipo: es la forma de comunicar al compilador qué funciones existen. La segunda, que validar los datos que ingresa el usuario no es un extra: el filtro de tres líneas fue lo que impidió que el programa dara resultados que no existen. La tercera, que las constantes son una forma barata de protegerse contra los errores de los demás.

También comprobé que un programa bien armado es más fácil de leer, de probar y de modificar. No tuvo errores de compilación y los tres resultados que esperaba salieron correctos, así que el camino funcionó.

La lección principal me la llevo conmigo: antes de escribir código conviene pararse a pensar la estructura. Dividir el problema antes de implementarlo sale siempre más barato que arreglar después un bloque largo. Esta manera de trabajar sirve para cualquier lenguaje y para cualquier tamaño de programa.

Referencias

Actividad 3: Enfoque modular y diseño Top-Down [Documento institucional]. (2026).

E.E.S.T. N.º 1 «Eduardo Ader», Vicente López.

Apéndice A

Código fuente completo del programa (src/main.cpp)

```
/*
   ================================================================
   Materia: Laboratorio de Programación (LPR)
   E.E.S.T. N° 1 "Eduardo Ader" – Vicente López
   Curso: 5° Año
   Profesor: Prof. York Elías Mansilla Muñoz
   Estudiante: CHEHEID MALEK
   Archivo: main.cpp
   Objetivo: Calcular el area de un circulo mediante funciones modulares
             y simular una diana de tiro en ASCII.

   Actividad 3: Enfoque Modular y Diseño Top-Down.
   ================================================================
*/

#include <iostream>

using namespace std;

// ================================================================
// 1. PROTOTIPO DE LA FUNCIÓN (Firma de la función)
// Le avisamos al compilador la existencia del módulo de cálculo antes del main
// ================================================================
double calcularAreaCirculo(double radio);

int main() {
    double radioEstudiante;

    // ================================================================
    // REGLA OBLIGATORIA (EVITA EL PLAGIO):
    // Modifiquen la salida de pantalla agregando su Nombre y Apellido reales.
    // ================================================================
    cout << "=====================================================" << endl;
    cout << "  CALCULADORA DE AREA MODULAR - ESTUDIANTE: CHEHEID MALEK  " << endl;
    cout << "=====================================================" << endl;

    cout << "=> Ingrese el radio del circulo/diana (en cm): ";
    cin >> radioEstudiante;

    // FILTRO DE CONSISTENCIA (Validación del dato geométrico)
    if (radioEstudiante <= 0) {
        cout << "[ERROR] El radio debe ser un valor positivo y mayor a cero." << endl;
        return 1; // Salida controlada del sistema indicando error en consola
    }

    // ================================================================
    // 2. LLAMADA / INVOCACIÓN DE LA FUNCIÓN
    // Convocamos al módulo 'calcularAreaCirculo' pasándole 'radioEstudiante'
    // y guardamos el resultado en una nueva variable decimal llamada 'areaFinal'.
    //
    //  SINTAXIS DE REFERENCIA (EJEMPLO GUÍA):
    //    Si quisiéramos llamar a una función 'calcularIva' que recibe 'precio'
    //    y guardar el resultado en 'ivaTotal', escribiríamos:
    //         double ivaTotal = calcularIva(precio);
    // ================================================================
```

```
    // Bloque 2: la invocación devuelve un double que se almacena en 'areaFinal'.
    double areaFinal = calcularAreaCirculo(radioEstudiante);

    // ========================================================================
    // 3. SALIDA DE DATOS
    // Muestren en pantalla el área calculada utilizando 'areaFinal'.
    //
    //  SINTAXIS DE REFERENCIA (EJEMPLO GUÍA):
    //    Para mostrar texto y variables combinadas en pantalla:
    //         cout << "El total es: " << costoTotal << " pesos." << endl;
    // ========================================================================

    // Bloque 3: 'areaFinal' se imprime junto al texto y el radio original.
    cout << "Radio ingresado: " << radioEstudiante << " cm" << endl;
    cout << "Area del circulo: " << areaFinal << " cm^2" << endl;

    // ========================================================================
    // BONUS GAMIFICACIÓN: RENDER DE LA DIANA EN CONSOLA
    // Dependiendo del radio que ingresaron, la consola simulará un blanco
    // de tiro correspondiente al tamaño calculado.
    // ========================================================================
    cout << "\n[SISTEMA] Dibujando escala de la Diana en la RAM..." << endl;
    if (radioEstudiante <= 5.0) {
        cout << "       .---.       " << endl;
        cout << "      /  X  \\     -> [DIANA MINI / COMPACTA]" << endl;
        cout << "      \\  *  /     " << endl;
        cout << "       '---'       " << endl;
    } else if (radioEstudiante <= 12.0) {
        cout << "       .---.       " << endl;
        cout << "     / .---. \\     " << endl;
        cout << "    | /  O  \\ |   -> [DIANA ESTÁNDAR DE TIRO]" << endl;
        cout << "    | \\  *  / |    " << endl;
        cout << "     \\ '---' /     " << endl;
        cout << "       '---'       " << endl;
    } else {
        cout << "       .---.       " << endl;
        cout << "     / .---. \\     " << endl;
        cout << "    | / .-. \\ |    " << endl;
        cout << "    | |  X  | |   -> [DIANA GIGANTE DE COBERTURA]" << endl;
        cout << "    | \\ '-' / |    " << endl;
        cout << "     \\ '---' /     " << endl;
        cout << "       '---'       " << endl;
    }

    cout << "======================================================" << endl;
    return 0; // Finalización exitosa de Windows (Código 0)
}

// ========================================================================
// 4. DEFINICIÓN / DESARROLLO DE LA FUNCIÓN
// Implementen la fórmula matemática de la geometría utilizando variables
// locales y la constante de pi locked en memoria.
//
//  SINTAXIS DE REFERENCIA (EJEMPLO GUÍA):
//    Para calcular un valor dentro de la función y retornarlo:
//         double total = base * altura;
//         return total;
// ========================================================================
double calcularAreaCirculo(double r) {
    const double PI = 3.1415926535;
```

```
    // Bloque 4: la fórmula del área de un círculo es A = PI * r^2.
    // 'r' es una variable LOCAL: vive solo dentro de este módulo y no
    // existe para el resto del programa (encapsulamiento del dato).
    // 'PI' es una constante: el compilador bloquea su escritura en RAM.
    double area = PI * r * r;

    // Se devuelve el resultado al programa principal (main).
    return area;
}
```